PioTask — Validación de teoría y arquitectura

Desarrollo de Aplicaciones I · UADE · Estado del código en `develop`

Documento para que el grupo revise si lo implementado coincide con la preentrega (secciones 7 a 17) y con la teoría de la materia. Cada afirmación remite a una clase o a un test que se puede abrir. Al final hay una lista de puntos que hay que decidir o corregir.

1. Resumen

Dato	Valor
Qué es	App Android que guarda todas las tareas y muestra solo las que necesitan atención: la Rama (despiertas) y el Nido (dormidas y pospuestas).
Stack	Kotlin, Jetpack Compose (Material 3), Navigation Compose, ViewModel y StateFlow, Room, DataStore, Hilt. minSdk 26, compileSdk 37.
Tamaño	70 archivos Kotlin en `main` (≈4250 líneas) y 746 líneas de tests.
Tests	48 tests unitarios, 0 fallas. Ninguno de Room, de ViewModels ni de UI.
Permisos	Ninguno: el manifiesto no declara `uses-permission` (sin red, sin notificaciones).
Hecho	Dominio, datos, onboarding, Rama, Nido, Detalle, Nueva/Editar tarea, Ajustes, hojas de avance y posponer, deshacer.
Pendiente	Widget con Glance, reevaluación diaria con WorkManager, tests de Room y de UI, validación con usuarios, caso de negocio.

2. Teoría aplicada

Concepto	Qué dice	Dónde está en PioTask
Clean Architecture: regla de dependencia	Las dependencias de código apuntan hacia adentro. El dominio no conoce frameworks.	`domain/` es Kotlin puro. El workflow de CI falla si algún archivo de esa carpeta importa `android` o `androidx` (`.github/workflows/ci.yml`).
Inversión de dependencias (la D de SOLID)	Lo de alto nivel define la interfaz; lo de bajo nivel la implementa.	Las interfaces `TareaRepository`, `CategoriaRepository`, `ActividadRepository` y `PreferenciasRepository` están en `domain/repository`. Las implementaciones están en `data/repository` y se enlazan con `@Binds` en `di/RepositoryModule`.
Responsabilidad única (la S)	Una clase, una razón para cambiar.	Un caso de uso por acción del usuario: `RegistrarAvanceUseCase`, `PosponerTareaUseCase`, `CompletarTareaUseCase`, etc. (14 casos de uso).

Segregación de interfaces (la I)	Nadie depende de lo que no usa.	`ActividadRepository` es de solo lectura; las escrituras de actividad van por `TareaRepository`.
MVVM y flujo unidireccional	La vista dibuja un estado y emite eventos; el ViewModel lo produce.	Cada pantalla tiene su ViewModel con `StateFlow<UiState>` (`RamaViewModel`, `NidoViewModel`, `DetalleViewModel`). Las pantallas Compose no tienen reglas de negocio.
Fuente única de verdad (offline-first)	Un solo lugar es dueño de los datos; la UI lo observa.	Room expone `Flow` (`TareaDao.observarActivas`); cualquier cambio vuelve a calcular la Rama sin que la pantalla lo pida.
Estado derivado, no persistido	Lo que se puede calcular no se guarda, para no desincronizarse.	El estado Dormido/Despierto/Reclamando no tiene columna: `CalcularEstadoAtencionUseCase` lo deriva de deadline, última actividad, posposición y fecha actual.
Tecnología calma (Weiser y Brown)	La información pasa de la periferia al centro solo cuando hace falta.	El Nido es la periferia y la Rama el centro. No hay notificaciones: el aviso es el widget (pendiente de implementar).
Atomicidad	Un cambio compuesto se aplica entero o no se aplica.	`TareaRepositoryImpl.guardarConActividad` y `revertir` usan `database.withTransaction`.
Determinismo y testeabilidad	El tiempo y los datos externos se inyectan.	El `Clock` se inyecta (`di/TiempoModule`); los tests usan un reloj fijo y repositorios en memoria (`Fakes.kt`) en lugar de mocks.
Manejo explícito de errores	Los fallos esperables son valores, no excepciones sueltas.	Los casos de uso de escritura devuelven `Resource<T>` con un mensaje apto para la UI (`domain/usecase/Resultado.kt`).

3. Arquitectura

Presentación · `presentation/`

Pantallas Compose, ViewModels, componentes comunes (pájaros, hojas, botones). Observa `StateFlow`; envía eventos a los casos de uso.

▼ la presentación depende del dominio

Dominio · `domain/` **(Kotlin puro)**

Modelos (`Tarea`, `Categoria`, `Criterios`, `Actividad`, `EstadoAtencion`), 14 casos de uso e interfaces de repositorio.

▲ la capa de datos depende del dominio: implementa sus interfaces

Datos · `data/`

Entidades y DAOs de Room, DataStore, mappers entidad↔modelo e implementación de los repositorios.

Transversales: `core/` (`Resource`, `UiState`, tema, rutas) y `di/` (módulos de Hilt: base de datos, DataStore, repositorios, reloj). El dominio no depende de ninguna de las otras dos capas. Es un único módulo Gradle (`:app`); las capas se separan por paquete y la regla se hace cumplir con CI.

3.1 Flujo de datos: registrar un avance

Es el flujo del diagrama 17.3 de la preentrega, con las clases reales:

1. `HojaAvance` (Compose) → el usuario guarda 60 % → `DetalleViewModel.registrarAvance(60)`.
2. `RegistrarAvanceUseCase` valida 0..100, busca la tarea (`tareaPendiente`: debe existir y no estar completada) y arma la tarea nueva con `progreso` y `ultimaActividad = ahora`.
3. `TareaRepository.guardarConActividad(tarea, actividad AVANCE)` → `TareaRepositoryImpl` abre una transacción: `UPDATE tarea` + `INSERT actividad`.
4. Room invalida la tabla y `TareaDao.observarActivas()` emite la lista nueva.
5. `ObtenerTareasEnAtencionUseCase` combina tareas y criterios, calcula estado y puntaje de cada una y reparte Rama / "+N" / Nido.
6. `RamaViewModel.uiState` emite; `RamaScreen` se recompone: el pájaro se duerme o cambia el motivo.
7. El caso de uso devuelve un `AccionDeshacible`; si el usuario toca "Deshacer", `DeshacerAccionUseCase` restaura la tarea anterior y borra la actividad, también en una transacción.

> **Diferencia con el diagrama 17.3:** el último paso del diagrama ("pedir actualización del widget") todavía no existe, porque el widget está pendiente.

3.2 Modelo de datos (Room, base `pio.db`, versión 1)

Tabla	Columnas	Relación
`categorias`	id, nombre, colorHex	1 categoría → N tareas. Borrado **RESTRICT**: no se puede eliminar con tareas.
`tareas`	id, titulo, categoriaId, progreso (0–100), deadline, pospuestaHasta, ventanaDeadlineDias, ventanaInactividadDias, creadaEn, ultimaActividad, completadaEn	N → 1 categoría. Índices en categoriaId y completadaEn.
`actividades`	id, tareaId, fecha, tipo (AVANCE / POSPUESTA / COMPLETADA), progreso	N → 1 tarea. Borrado **CASCADE**.

Fechas: `Instant` como milisegundos y `LocalDate` como día de época (`Converters`), para poder comparar en SQL. Preferencias en DataStore: criterios globales (ventana de deadline, ventana de inactividad, límite visible), onboarding visto y última categoría usada.

4. Reglas de atención

Fuente: `docs/reglas-atencion.md`. Deadline e inactividad son dos relojes independientes; el estado final es el mayor de los dos.

Reloj	Dormido	Despierto	Reclamando
Deadline (ventana 7)	Sin deadline, o faltan más de 7 días	Faltan de 7 a 3 días	Faltan 2 días o menos, o está vencida
Inactividad (ventana 30)	Menos de 12 días sin avance	De 12 a 29 días	30 días o más

Parámetros internos: `diaReclamo = 30 % de la ventana de deadline` (redondeado hacia abajo: 7 → 2) y `diaDespierto = 40 % de la ventana de inactividad` (redondeado hacia arriba: 30 → 12). Redondear hacia arriba evita que una ventana de 1 día deje todo siempre despierto.

Puntaje (ordena dentro de un mismo estado)

```
Pd = presión de deadline   = 0 si no hay deadline; si hay, (ventana − días restantes) /
ventana, limitado a 0..1
Pi = presión de inactividad = días sin avance / ventana de inactividad, limitado a 0..1
Tr = trabajo restante       = (100 − progreso) / 100

puntaje = 100 × (0,6 × Pd × (0,5 + 0,5 × Tr) + 0,4 × Pi)
```

El trabajo restante solo cambia el orden dentro de un estado, nunca el estado. Ejemplo documentado y probado: una tarea que vence en 2 días y lleva 5 sin avance da **45** puntos con 20 % de progreso y **30** con 90 %.

Orden y límite

1. Estado: Reclamando antes que Despierto.
2. Puntaje, de mayor a menor.
3. Deadline más cercano; las tareas sin deadline van al final.
4. Id, para que el orden sea estable.

La Rama muestra las primeras N y el resto se cuenta en "+N tareas más requieren atención". **Regla de seguridad:** una tarea Reclamando nunca queda en el Nido (hay un test que lo verifica).

Posposición

No es un cuarto estado, es una condición de visibilidad: hasta la fecha elegida la tarea está en el Nido con el pájaro dormido. Posponer no cuenta como avance (no toca `ultimaActividad`). Si por deadline la tarea entra en Reclamando durante la posposición, vuelve a la Rama; esto incluye posponer una tarea que ya está en ese estado.

5. Trazabilidad con la preentrega

5.1 Requisitos funcionales

Req.	Estado	Qué hay y qué falta
RF01	Parcial	Título, categoría y fecha límite; la tarea nace con `ultimaActividad = creación` y su estado se calcula (`GuardarTareaUseCase`, con tests). **Falta** el bloque "Más opciones" (progreso inicial, "Posponer hasta", ventanas por tarea): el modelo y el caso de uso ya aceptan esos campos, pero la pantalla no los pide.
RF02	Parcial	Estado, puntaje, orden, límite N, "+N" y motivo en cada tarjeta (`ObtenerTareasEnAtencionUseCase`, 9 tests). Se recalcula al abrir la app y tras cada acción porque la Rama observa los datos. **Falta** la reevaluación una vez al día en segundo plano (WorkManager).
RF03	Cumple	Registrar avance (hoja con atajos +10 %, +25 % y "mismo %"), Posponer (mañana, 3 días, 1 semana, fecha exacta) y Completar, desde la tarjeta y desde el Detalle. "Deshacer" de 5 s para completar y posponer. 11 tests de casos de uso.
RF04	Parcial	Límite visible 3/5/7, ventana de deadline y de inactividad globales, categorías con nombre y color (crear, editar, eliminar con reasignación). Al guardar, la Rama se recalcula sola (test). **Falta** la ventana por tarea en la UI y la actualización del widget.

5.2 Requisitos no funcionales

Req.	Estado	Evidencia o brecha
RNF01 Offline	Cumple	Sin permisos de red; todo pasa por Room y DataStore.
RNF02 Alta en ≤3 interacciones	Cumple	"+", escribir el título, "Guardar tarea". La última categoría usada queda preseleccionada.
RNF03 Rama <1 s con 200 tareas	Sin medir	No hay medición. Hay que cargar 200 tareas y medir.
RNF04 Accesibilidad de estado	Parcial	Cumple: forma distinta por estado y descripción para TalkBack en cada pájaro; la categoría siempre lleva nombre además del color. **Falta** la etiqueta de texto (DORMIDO / DESPIERTO / RECLAMANDO) en las tarjetas de Rama y Nido; hoy aparece solo en el Detalle y el onboarding. Los wireframes tampoco la muestran en la tarjeta.
RNF05 Áreas táctiles ≥48 dp	Parcial	Los botones "Avancé" y "Completar" de la tarjeta miden 44 dp, que es lo que dibuja el Figma, pero la preentrega pide 48 dp. Tampoco llegan a 48 dp los chips de filtro (36 dp), los atajos de la hoja de avance (40 dp), las opciones de categoría del formulario (44 dp) y los selectores de Ajustes (40 dp). Los botones principales (Guardar, Completar tarea, Posponer) miden 52 dp o más.
RNF06 Confiabilidad	Parcial	Room con transacciones. No hay test que simule cierre o reinicio.

RNF07 Batería	Pendiente	Depende de WorkManager, sin implementar. No hay procesos periódicos en la app.
RNF08 Privacidad	Cumple	Manifiesto sin `uses-permission`; datos solo en el dispositivo.

6. Pruebas

Archivo	Tests	Qué verifica
`CalcularEstadoAtencionUseCaseTest`	13	Límites de cada estado por deadline e inactividad, el mayor de los dos relojes, ventana por tarea, posposición y su excepción.
`CalcularPuntajeAtencionUseCaseTest`	5	Fórmula documentada, peso del trabajo restante, tope en 100.
`ObtenerTareasEnAtencionUseCaseTest`	9	Reparto Rama / "+N" / Nido, orden, desempate, límite, cambio de fecha y de criterios, ninguna tarea Reclamando oculta.
`AccionesSobreTareaUseCasesTest`	11	Guardar, avance, posponer, completar y deshacer; validaciones; posponer no cuenta como avance.
`CategoriasYAjustesUseCasesTest`	7	Validación de categorías, borrado con reasignación, categorías sugeridas, criterios (3/5/7), onboarding.
`MappersTest`	3	Ida y vuelta entidad↔modelo y conversores de fechas.
Total	**48**	

No cubierto: consultas y transacciones de Room (requieren dispositivo), ViewModels, pantallas Compose y las hojas. Las pantallas se probaron a mano en un emulador Pixel 9a: alta con fecha, estado Reclamando, completar con deshacer, posponer, editar, eliminar, onboarding de primer uso y Ajustes.

7. Decisiones de diseño tomadas

Decisión	Motivo
Cuatro repositorios en lugar de dos	La sección 12 nombra `TareaRepository` y `PreferenciasRepository`; las categorías y las actividades necesitan su propio acceso. `ActividadRepository` es de solo lectura para que la escritura quede en la misma transacción que la tarea.
Casos de uso que devuelven `Resource`	La UI muestra el mensaje de error sin manejar excepciones.
Avisos con "Deshacer" en un bus global	Si el aviso viviera en cada pantalla se perdería al volver desde el Detalle.
El pájaro se elige por cercanía de color con las 3 categorías del diseño	Las categorías las define el usuario; no se puede mapear por nombre.

Un único módulo Gradle	El alcance no justifica varios; la regla de dependencias se hace cumplir en CI.
Tema solo claro	El Figma no define modo oscuro.
La Rama muestra "cargando" al volver, no el último estado	El flujo se detiene a los 5 s sin pantalla; mostrar el último valor enseñaba datos viejos.

8. Puntos a validar o corregir

Para decidir entre todos o con la cátedra

1. **Fórmula del puntaje.** Los pesos (0,6 deadline, 0,4 inactividad, base 0,5 del trabajo restante) siguen sin aprobación ni validación con usuarios; es la observación del profe.
2. **12 o 14 días.** La sección 9 de la preentrega y el código usan 12 días (40 % de 30) para despertar por inactividad; la descripción del componente "Pájaro" en el Figma dice "14+ días". Hay que unificar.
3. **Posponer una tarea que ya reclama.** Hoy no se puede ocultar; la hoja muestra un aviso genérico y no avisa de esta excepción, como pide `docs/reglas-atencion.md` y el aviso del frame 11 del Figma.
4. **¿Llegar a 100 % completa la tarea?** Sigue abierto en `docs/reglas-atencion.md`.
5. **Medianoche con la app abierta.** La Rama no recalcula hasta que cambian los datos o se vuelve a abrir. El widget con WorkManager lo resolverá para la pantalla de inicio, pero no dentro de la app.

Brechas con la preentrega que conviene corregir

1. Etiqueta de texto del estado en las tarjetas (RNF04).
2. Controles de 36 a 44 dp contra los 48 dp pedidos (RNF05); el Figma dibuja varios con esa medida, así que hay que decidir si se cambia el diseño o el requisito.
3. "Más opciones" de Nueva tarea y ventanas por tarea en RF01 y RF04.
4. Medir RNF03 con 200 tareas.
5. Tests de Room, de ViewModels y de las pantallas clave.

9. Pendiente

1. **Widget con Glance** (3 variantes del frame 12) que reutiliza `ObtenerTareasEnAtencionUseCase`.
2. **Reevaluación diaria con WorkManager**, una vez por día, que actualiza el widget.
3. Validación con usuarios: ¿reduce el tiempo de organización y los olvidos frente a una lista convencional? ¿Los reaparecidos saturan?
4. Caso de negocio: diferencia suficiente para cambiar de app y cómo se sostendría el producto.
5. Prueba de cambios de fecha, posposiciones y actualización del widget, verificando que ninguna tarea importante quede oculta (entrega final).